

The Evolution of AI Agents

From Memory Architecture to Self-Improving Agents

Zhixuan Tan

Student ID:225040506

School of Data Science

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Large language model (LLM) agents have rapidly evolved from single-turn text generators into interactive systems that can use tools, maintain conversations, and act in external environments. However, as agents are deployed in longer and more realistic settings, their limitations are no longer only about reasoning ability in a single prompt. A capable agent must also remember past interactions, organize accumulated experience into reusable knowledge, and improve its behavior through continued use. This report studies this evolution through three representative systems: MemGPT [3], Agentic Context Engineering (ACE) [10], and OpenClaw-RL [6]. MemGPT treats memory as a system design problem and introduces operating-system-inspired virtual context management for LLM agents. ACE moves beyond raw memory by transforming experience into structured, evolving playbooks that improve the external context used by the agent. OpenClaw-RL goes further by treating interaction itself as a training signal, enabling policies to improve through reinforcement learning and on-policy distillation across personal and general agent environments. Together, these systems illustrate a broader technical trajectory: from managing memory, to evolving context, to updating behavior. This report analyzes the motivation, design principles, empirical implications, and limitations of each system, and concludes by discussing open challenges in building stable, safe, and continuously improving AI agents.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation: Beyond Single-Turn Reasoning	1
1.2 From Limited Context to Continual Agent Evolution	2
1.3 Practical Symptoms of Long-Horizon Agent Failure	3
1.4 Three Central Questions	4
1.5 Scope and Contributions of This Report	4
1.6 Organization of the Report	5
2 Background and Related Work	7
2.1 Large Language Model Agents	7
2.2 Context Windows and Long-Horizon Interaction	8
2.3 Memory-Augmented LLM Agents	8
2.4 Context Engineering and Prompt Evolution	9
2.5 Reinforcement Learning for Interactive Agents	10
2.6 External vs Internal Adaptation	10
2.7 Why These Three Papers Form a Natural Progression	11
2.8 Summary	11
3 Memory Architecture: MemGPT	13
3.1 Motivation: The Limits of Fixed Context Windows	13
3.2 Design Rationale of the Operating-System Analogy	14
3.3 Memory Hierarchy in Detail	14
3.4 Why Explicit Memory Operations Matter	15
3.5 Memory Update and Retrieval	16
3.6 Empirical Evidence: Deep Memory Retrieval	17

3.7	Benefits and Potential Failure Cases	17
3.8	Comparison with Naive Long-Context Baselines	18
3.9	Summary and Evaluation	18
4	Context Evolution: Agentic Context Engineering	20
4.1	Motivation: Why Raw Memory Is Not Enough	20
4.2	Context Collapse	21
4.3	Why Summarization Alone Fails	21
4.4	The Generator–Reflector–Curator Framework	22
4.5	From Trajectories to Reusable Playbook Items	22
4.6	Structured Playbooks as External Strategy	23
4.7	Empirical Results and Limitations	24
4.8	Playbook Growth, Redundancy, and Conflict	25
4.9	Summary and Evaluation	25
5	Policy Evolution: OpenClaw-RL	27
5.1	Motivation: From Evolving Context to Evolving Behavior	27
5.2	Why External Context Eventually Hits a Limit	28
5.3	Train-While-Serving Architecture	28
5.4	Next-State Signals as a General Learning Interface	29
5.5	Learning Signals from Interaction	30
5.6	Learning Across Agent Environments	31
5.7	Empirical Implications and Personalization	32
5.8	Safety, Stability, and Deployment Challenges	32
5.9	Summary and Evaluation	33
6	Comparative Analysis	35
6.1	What Is Improved: Memory, Context, or Policy	35
6.2	Knowledge Representation Across the Three Systems	36
6.3	Adaptation Mechanisms	36
6.4	External vs Internal Adaptation	37
6.5	When Should an Agent Update Memory, Context, or Policy?	37
6.6	Critical Comparison	38
6.7	Lessons Learned from the Three Papers	39
6.8	Stability as the Central Bottleneck	40

6.9	Limitations and Open Challenges	40
6.10	Summary	41
7	Conclusion and Future Work	42
7.1	Conclusion	42
7.2	Toward a Unified Agent Architecture	43
7.3	Implications for Future Agent Benchmarks	43
7.4	Implications for Real-World Deployment	44
7.5	Future Work	44
7.6	Final Perspective on Hybrid Agent Systems	45

List of Figures

1.1	Evolution path of AI agents considered in this report. The progression moves from a standard LLM with limited context to memory-managed, context-evolving, and self-improving agents. Each stage expands the agent’s ability to use experience: from maintaining only the current prompt, to retrieving long-term memory, to organizing experience into an evolving playbook, and finally to improving behavior through interaction.	3
3.1	MemGPT memory architecture. The LLM’s finite context window is divided into system instructions, working context, and a FIFO queue, while older or less immediately relevant information is moved into external archival and recall storage. Through the function executor and queue manager, the agent can explicitly read, write, and retrieve memory, turning memory management into a system-level mechanism rather than relying on a single flat prompt.	15
4.1	Context collapse under repeated monolithic rewriting. Although summarization can reduce the number of tokens in the context, repeated rewriting may remove useful procedural details and cause performance to drop. This motivates ACE’s argument that good context should not merely be shorter, but should preserve reusable knowledge in a structured and incrementally maintained form.	21
4.2	Agentic Context Engineering framework. The generator uses the current context playbook to solve tasks and produce trajectories; the reflector analyzes these trajectories to extract useful insights; and the curator converts these insights into delta context items that update the playbook. This loop turns raw interaction experience into structured and reusable external strategy.	23

5.1	OpenClaw-RL system architecture for train-while-serving agents. Personal and general agents interact with users, tools, and environments through environment servers, while the RL server asynchronously collects interaction data and updates the policy. This design allows deployed agents to continue serving requests while their behavior is improved through reinforcement learning and reward modeling. . .	29
5.2	Learning signals used by OpenClaw-RL. The framework supports binary rewards from user or environment feedback, directional supervision through on-policy distillation, and step-wise reward estimation for long trajectories. These complementary signals allow interaction data to provide both outcome-level evaluation and more detailed guidance for policy improvement.	30
5.3	Performance improvement of OpenClaw-RL across multiple agent settings. The curves show that reinforcement learning from interaction can improve agents in terminal, GUI, software engineering, and tool-call environments. This supports the broader claim that agent behavior can be optimized using feedback generated during deployment rather than only through static pretraining or prompt engineering.	32
5.4	Personalization example in OpenClaw-RL. After several interactions, the agent adapts to different user preferences: the student setting encourages responses that sound less AI-like, while the teacher setting encourages more specific and friendly feedback. This example illustrates how interaction-based learning can improve not only task correctness, but also user-specific behavior and communication style.	33

List of Tables

2.1	External and Internal Adaptation in Agent Systems	11
3.1	Memory Layers in MemGPT	15
3.2	Deep Memory Retrieval Results Reported for MemGPT	17
4.1	Desirable Properties of Playbook Items	24
4.2	Summary of ACE Performance Improvements	24
5.1	Next-State Signals Across Agent Settings	31
6.1	Comparison of Three Agent Evolution Systems	36
6.2	A Simple Decision Framework for Agent Updates	38

Chapter 1

Introduction

1.1 Motivation: Beyond Single-Turn Reasoning

The first generation of large language model applications mostly treated the model as a powerful single-turn problem solver. A user provides an instruction, the model generates an answer, and the interaction ends. Under this view, the main question is whether the model can reason correctly within the current prompt. Evaluation also tends to follow this framing: models are tested on static benchmarks, isolated questions, and independent tasks. This paradigm has been extremely useful for measuring general language understanding, mathematical reasoning, coding ability, and tool-use capability [1, 2, 7].

However, modern AI agents are increasingly used in settings that are not single-turn and not independent. A personal assistant may interact with the same user over many days or months. A coding agent may inspect files, run tests, observe error messages, modify code, and revise its solution over many steps. A terminal or GUI agent may need to track environmental state, past actions, and user preferences. In these settings, the agent’s success depends not only on whether the base LLM is intelligent in one isolated prompt, but also on whether the overall system can manage experience over time.

This shift changes the central problem. The question is no longer simply, “Can the model solve this task?” A more realistic question is, “Can the agent remember what matters, organize its accumulated experience, and improve after interacting with the world?” This is the motivation behind the evolution studied in this report. Agents are not only becoming better reasoners; they are also becoming systems with memory, external knowledge structures, and learning loops.

In practice, this problem is easy to observe. When the context is short and clean, an LLM agent often appears highly capable. It can answer questions, follow

instructions, and reason through simple tasks. But as more information is added—conversation history, retrieved documents, tool outputs, prior attempts, user preferences, and task-specific rules—the agent may become less reliable. Important information may be buried in long context. Irrelevant information may distract the model. Repeated summaries may lose crucial details. Long prompts may contain overlapping or contradictory instructions. In other words, simply giving the model more context does not automatically produce a better agent.

This motivates three core needs for future AI agents. First, they need memory: the ability to store and retrieve information beyond a limited context window. Second, they need organization: the ability to transform raw experience into structured and reusable guidance. Third, they need improvement: the ability to update behavior through interaction rather than remaining static after deployment. These three needs correspond to the three systems studied in this report: MemGPT, Agentic Context Engineering, and OpenClaw-RL.

1.2 From Limited Context to Continual Agent Evolution

The evolution path can be summarized as a movement from limited context to self-improving systems. A standard LLM operates within a finite context window. Even if this window becomes very large, it remains limited, expensive, and difficult to use effectively. The first step is therefore memory management: deciding what should remain in active context, what should be stored externally, and when the agent should retrieve older information. MemGPT represents this stage by treating the LLM context window as a scarce resource that must be managed similarly to memory in an operating system.

The second step is context evolution. Once agents can store information, the next problem is how to make stored experience useful. A raw log of past interactions is usually not the ideal form of knowledge. It may contain noise, repetition, obsolete facts, and task-specific details that do not transfer. What the agent needs is not just memory, but an evolving external playbook: a structured collection of strategies, rules, templates, and troubleshooting knowledge. ACE represents this stage by arguing that context should evolve incrementally through generation, reflection, and curation.

The third step is policy evolution. Even a well-structured playbook remains external to the model. The underlying model parameters are still mostly fixed. The model must read the context, select relevant information, and apply it correctly. As

the external context grows, the model may fail to attend to the right parts or may misapply rules. This suggests that some improvements should be internalized into the policy itself. OpenClaw-RL represents this stage by treating interaction as a source of reinforcement learning and distillation signals. Instead of only changing the prompt or memory, the system updates the model behavior.

These three stages are not mutually exclusive. A future agent may use all of them together. It may maintain long-term memory, organize experience into external skills, and train its policy from interaction data. The contribution of this report is to show how these stages relate to one another and how each system solves a different part of the broader continual agent problem.

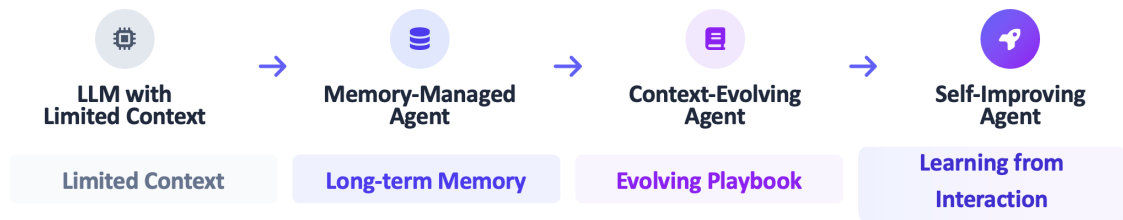


Figure 1.1: Evolution path of AI agents considered in this report. The progression moves from a standard LLM with limited context to memory-managed, context-evolving, and self-improving agents. Each stage expands the agent’s ability to use experience: from maintaining only the current prompt, to retrieving long-term memory, to organizing experience into an evolving playbook, and finally to improving behavior through interaction.

1.3 Practical Symptoms of Long-Horizon Agent Failure

The shift from single-turn tasks to long-horizon interaction is not merely a theoretical concern. In practical use, several recurrent symptoms appear when an agent is deployed over longer time horizons. The first is *forgetting*. The agent may fail to recall a user preference, an earlier decision, or a previously established constraint. The second is *context dilution*. The agent receives all prior information, but the most important details are no longer salient. The third is *instruction conflict*. As more context accumulates, the agent may face overlapping or contradictory guidance and struggle to prioritize the relevant rule.

A related symptom is *poor state maintenance*. In interactive environments, the agent must often keep track of current files, environment state, prior tool outputs, and tentative hypotheses. Losing track of any of these can break long trajectories even when each local step looks individually reasonable. Another symptom is *low*

transfer from experience. The agent may encounter a recurring class of mistakes, but unless experience is transformed into reusable knowledge, it repeats the same pattern.

These symptoms help explain why longer context alone is insufficient. The problem is not only capacity; it is also organization and control. A long prompt can contain more information, yet still fail to provide the right information in the right form. For this reason, the evolution from limited-context models to self-improving agents should be understood as an architectural transition rather than a simple scaling trend.

1.4 Three Central Questions

This report is organized around three central questions. The first question is: *How should agents remember?* This concerns memory architecture. Once the active context window is treated as a limited working memory, the system must decide what remains visible, what is stored externally, and how external information is retrieved. MemGPT directly addresses this question.

The second question is: *How should context evolve?* Memory alone is not enough, because raw history is rarely the best format for future decision making. Experience often must be abstracted into strategies, rules, or troubleshooting guidance. ACE addresses this question by treating context as an evolving playbook rather than a monolithic prompt or repeatedly compressed summary.

The third question is: *How can agents improve from interaction?* Even a strong external playbook still depends on a mostly fixed underlying policy. If the agent repeatedly encounters feedback from users, tools, environments, or tests, this feedback can be used to improve the policy itself. OpenClaw-RL addresses this question by converting next-state signals into reinforcement learning and distillation signals.

Taken together, these three questions define the main conceptual path of the report: memory, organization, and improvement. They also motivate the broader thesis of this report: future AI agents should be studied as systems that manage and transform experience over time, rather than only as static models evaluated in isolation.

1.5 Scope and Contributions of This Report

This report is a survey-style and analysis-oriented final report. It does not propose a new algorithm, but it synthesizes three recent systems into a unified view of agent

evolution. I selected these three papers because they helped me understand agent improvement as a system-level problem rather than only a model-scaling problem. Before studying these systems, it is natural to think of stronger agents mainly as stronger models. After comparing them, I find that agent capability can also come from better memory management, better context organization, and better use of interaction feedback. This is why I organize the report around a progression from memory, to context, to policy improvement.

The report focuses on three representative papers:

1. **MemGPT: Towards LLMs as Operating Systems.** This system addresses the memory management problem by introducing virtual context management and hierarchical memory for LLM agents.
2. **Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models.** This system addresses the problem of context collapse and proposes structured evolving playbooks as an alternative to repeated monolithic summarization.
3. **OpenClaw-RL: Train Any Agent Simply by Talking.** This system addresses policy improvement by converting ongoing interaction into reinforcement learning and on-policy distillation signals.

The report makes three main analytical contributions. First, it explains how the three systems form a coherent technical progression: MemGPT externalizes memory, ACE externalizes strategy, and OpenClaw-RL updates behavior. Second, it compares their knowledge representations, adaptation mechanisms, and limitations. Third, it identifies open challenges for future agent systems, including stable continual learning, safe policy updates, feedback quality, and the balance between external context and internal model learning.

A personal goal of this report is to avoid treating the three papers as isolated summaries. Instead, I use them to develop my own view of agent evolution: future agents should not depend on a single mechanism such as longer context, larger memory, or reinforcement learning alone. A practical agent will likely need a layered architecture where different kinds of experience are handled at different levels.

1.6 Organization of the Report

The rest of the report is organized as follows. Chapter 2 provides broader background on large language model agents, long-horizon interaction, memory augmentation, context engineering, and reinforcement learning for interactive systems.

Chapters 3, 4, and 5 then study MemGPT, ACE, and OpenClaw-RL in depth. Each chapter examines the motivation, design choices, empirical claims, and limitations of one system.

Chapter 6 provides the main comparative analysis. It examines what each system improves, how knowledge is represented, which adaptation mechanism is used, and how memory, context, and policy updates relate to one another. Chapter 7 concludes the report by summarizing the main lessons and proposing a unified view of future hybrid agent architectures.

Chapter 2

Background and Related Work

2.1 Large Language Model Agents

Large language models were originally developed as general-purpose sequence models trained to predict and generate text. Their capabilities expanded significantly with instruction tuning, reinforcement learning from human feedback, tool use, and chain-of-thought style reasoning. As a result, LLMs are now widely used as the core decision-making component in AI agents.

An LLM agent is usually more than a language model. It is a system that combines the model with prompts, memory, tools, external environments, and control logic. For example, a tool-using agent may generate an action, call an API, observe the result, and then decide the next action. A coding agent may inspect a repository, run tests, and modify files. A personal assistant may retrieve user preferences and maintain a long-term profile. In each case, the LLM is embedded inside a larger loop.

This system-level view is important because the performance of an agent depends not only on the model weights, but also on the surrounding architecture. A stronger model can fail if it receives confusing context. A weaker model can perform surprisingly well if it is given well-structured instructions, relevant memories, and reliable feedback. Therefore, agent design is increasingly about the interaction between model, context, memory, tools, and learning signals.

Recent work has also shown that the boundary between inference and control is becoming thinner. Systems inspired by ReAct [7] combine reasoning and acting, while verbal reinforcement methods such as Reflexion [5] attempt to improve future decisions using language-level feedback. These developments suggest that the agent is best viewed as a decision system with multiple interacting substrates: model parameters, prompts, memories, external tools, and feedback loops.

2.2 Context Windows and Long-Horizon Interaction

The context window is the visible working memory of an LLM. All information that the model uses directly at generation time must be represented as tokens inside this window. This includes system instructions, user messages, conversation history, retrieved documents, tool outputs, and any external memory inserted into the prompt.

A limited context window creates an obvious problem for long-horizon interaction. If the conversation or task history becomes longer than the window, some information must be removed, summarized, or stored elsewhere. Even when the context window is large enough to include all information, the model may still struggle to use it effectively. Long context can introduce noise, reduce attention to important details, and increase the chance of instruction conflict.

A naive solution is to keep appending information. This works for short tasks but becomes inefficient and unstable in longer settings. Another naive solution is to repeatedly summarize the history. This reduces length but can destroy useful details. The central challenge is therefore not only how to store more information, but how to decide what information should be visible, what should be retrievable, and what should be converted into reusable knowledge.

This challenge motivates memory-augmented and context-engineered agents. Memory-augmented agents separate active context from long-term storage. Context-engineered agents modify the prompt or external context so that the model receives more useful guidance. Both approaches recognize that agent intelligence is not only located in the model parameters; it is also distributed across the system’s context and memory design.

2.3 Memory-Augmented LLM Agents

Memory-augmented agents extend the basic LLM interface by adding explicit storage and retrieval mechanisms. These mechanisms may be as simple as a vector store used for retrieval-augmented generation, or as structured as a multi-tier memory hierarchy in which some information is maintained as short-term working state and other information is archived for later retrieval.

A useful distinction is between *episodic* and *procedural* memory. Episodic memory stores facts about past events: what the user said, what tool output was observed, or what state the environment was in. Procedural memory stores knowledge

about how to act: which rule to apply, which failure pattern to avoid, or what troubleshooting routine to follow. Many simple memory-augmented systems handle the episodic part more easily than the procedural part. They can store and retrieve facts, but they do not automatically convert recurring experience into better future policy.

Another distinction is between *append-only* memory and *revisable* memory. Append-only logs are easy to implement, but they accumulate outdated or conflicting information. Revisable memory is harder because it requires the system to determine when a new fact supersedes an older fact, and how to preserve useful historical information while keeping current state consistent.

Memory augmentation is therefore not just a question of adding a database. It requires decisions about representation, retrieval, updating, and salience. MemGPT is representative because it elevates these decisions into an explicit system architecture rather than treating memory as a hidden implementation detail.

2.4 Context Engineering and Prompt Evolution

Prompt engineering is often understood as the manual design of instructions that make the model perform better. Context engineering is broader. It includes all mechanisms by which the external textual environment of the model is constructed, maintained, updated, or reorganized. This includes not only static prompt design, but also memory insertion, summary construction, playbook updates, retrieved exemplars, and structured guidance.

A key insight from recent agent work is that *better context is not always longer context*. A longer context can hold more information, but it can also weaken the signal-to-noise ratio. Information that is nominally available may not be practically usable, because it is poorly organized or buried under less relevant content. Thus, the problem of prompt evolution is also a problem of selection and abstraction.

Repeated summarization is an especially important case. It seems attractive because it limits context growth, but it may gradually erase the very information that makes the context useful. Procedural details are particularly vulnerable. A summary can retain a high-level task statement while losing the concrete rule or failure condition needed to act correctly. ACE is important precisely because it treats context evolution as a structured process rather than a repeatedly rewritten monologue.

2.5 Reinforcement Learning for Interactive Agents

Reinforcement learning has long been a framework for learning from sequential interaction. In classical RL, the agent acts in an environment, receives a reward, and learns a policy that maximizes expected return. Interactive language agents fit this framework only imperfectly. Their environments are more heterogeneous, rewards are often delayed or sparse, and some of the most useful feedback is not scalar but linguistic.

For this reason, RL for language agents often combines multiple sources of signal. A terminal agent may obtain clear success or failure signals from exit codes. A software engineering agent may receive test results, compiler errors, or diffs. A personal assistant may receive user replies, corrections, or preference feedback. These signals differ in precision and reliability, yet they all reflect consequences of action.

Another important distinction is between *evaluative* and *directive* feedback. Evaluative feedback says whether the action was good or bad. Directive feedback says how it should be improved. OpenClaw-RL is notable because it attempts to use both. In that sense, it occupies a space between classical reinforcement learning [4] and language-level improvement methods such as verbal reflection or correction [5].

2.6 External vs Internal Adaptation

One of the most important conceptual distinctions in this report is between external and internal adaptation. External adaptation changes the information given to the model at inference time. This includes memory retrieval, playbook updates, and prompt restructuring. Internal adaptation changes the model policy itself, usually by updating parameters or training a reward-conditioned behavior.

External adaptation is attractive because it is transparent, editable, and relatively easy to reverse. If a memory entry is wrong or a playbook rule becomes outdated, a human can inspect and change it. Internal adaptation is attractive because it can produce deeper, more fluent, and more consistent behavioral changes. Once the policy has internalized a pattern, the agent may no longer need explicit prompting to use it.

Neither form of adaptation dominates the other. Some knowledge is naturally external, such as a changing user profile or a document repository. Some knowledge may be better internalized, such as repeated preferences for action sequencing or communication style. The three systems in this report can therefore be seen as

occupying different points on the external–internal spectrum.

Table 2.1: External and Internal Adaptation in Agent Systems

Adaptation Type	Main Mechanism	Typical Strengths
External adaptation	Memory retrieval, context editing, play-book updates	Interpretable, editable, low-cost, reversible
Internal adaptation	Policy learning, reward-based updates, distillation	Deeper behavioral change, smoother execution, reduced prompt dependence

2.7 Why These Three Papers Form a Natural Progression

The three papers discussed in this report are useful not only because each is influential on its own, but because together they reveal a natural progression in agent design. MemGPT asks how the agent should *remember*. ACE asks how the agent should *organize* experience into reusable context. OpenClaw-RL asks how the agent should *improve* through interaction.

This progression is also a progression in the location of knowledge. In MemGPT, knowledge is stored mainly as retrievable memory. In ACE, knowledge becomes structured external strategy. In OpenClaw-RL, part of the improvement is internalized into policy behavior. The result is a ladder of increasingly ambitious adaptation mechanisms.

This framing matters because it clarifies a frequent confusion in agent research. Not all agent failures require policy training, and not all failures can be solved by simply adding more context. Some failures are about missing state, some about missing procedure, and some about the inability to internalize repeated experience. Studying these three papers together makes these differences more visible.

2.8 Summary

This chapter positioned the rest of the report by reviewing large language model agents, long-horizon interaction, memory augmentation, context engineering, reinforcement learning for interactive systems, and the distinction between external and internal adaptation. The key conclusion is that the three papers analyzed later

should not be treated as isolated proposals. They describe different layers of a broader architectural transition in AI agents.

Chapter 3

Memory Architecture: MemGPT

3.1 Motivation: The Limits of Fixed Context Windows

MemGPT begins from a simple but fundamental limitation of LLM agents: the active context window is finite. In a short interaction, this limitation may not matter. The entire conversation can fit inside the prompt, and the model can directly condition on all relevant information. In a long interaction, however, the agent must deal with much more information than can be kept active at once.

This problem is especially important for two domains: long-document analysis and multi-session conversation. In long-document analysis, the document may exceed the context window of the underlying model. In multi-session conversation, the agent may need to remember information from previous sessions, such as user preferences, personal facts, or earlier decisions. Without a memory mechanism, the model either forgets this information or requires the user to repeat it.

The key insight of MemGPT is that this problem resembles memory management in traditional computer systems. A computer does not keep all data in the fastest memory at all times. Instead, it uses a hierarchy: registers, cache, main memory, and disk. The operating system manages movement between these layers so that the program experiences a larger virtual memory than the physical fast memory. MemGPT applies a similar idea to LLM context. The model’s prompt is treated as limited active memory, while external stores provide longer-term memory.

This analogy is powerful because it shifts the design question. Instead of asking how to fit everything into the prompt, MemGPT asks how an agent can manage information across memory tiers. The prompt becomes a scarce working resource. External memory becomes a place to store information that may be useful later.

Function calls become the mechanism for reading and writing memory.

3.2 Design Rationale of the Operating-System Analogy

The operating-system analogy is not simply a metaphor for explaining the paper to readers. It also serves as a design principle. In an operating system, the programmer does not manually place every byte into every memory tier at every step. Instead, the system provides abstractions that separate active use from longer-term storage. MemGPT attempts a similar separation for LLM agents.

This analogy is compelling for three reasons. First, it highlights scarcity. The active context window should be treated as a high-value and limited resource. Second, it highlights movement. Information does not need to remain active permanently; it can be archived and later retrieved when needed. Third, it highlights system control. Memory is not a by-product of prompting but a managed process with explicit operations.

By making the context window functionally closer to working memory and the external stores functionally closer to long-term storage, MemGPT reframes the design of long-horizon agents. The central problem becomes not prompt length, but memory policy: what enters, what leaves, what is stored, and what is recalled.

3.3 Memory Hierarchy in Detail

MemGPT divides memory into several components. The active prompt contains information that is immediately visible to the model. This includes system instructions, a working context, and recent messages. The working context functions like a scratchpad or editable memory block. It can contain important facts, current goals, and state variables. Recent messages are stored in a FIFO queue, so newer interactions remain visible while older ones may be moved out.

Beyond the active prompt, MemGPT uses external storage. Archival storage stores general long-term information, such as facts about the user or documents that may need to be retrieved later. Recall storage stores older conversation history. The model can explicitly search or retrieve from these stores when needed. This makes memory queryable rather than merely implicit.

The hierarchy matters because it separates different roles of information. System instructions define the persistent contract of behavior. Working context stores what

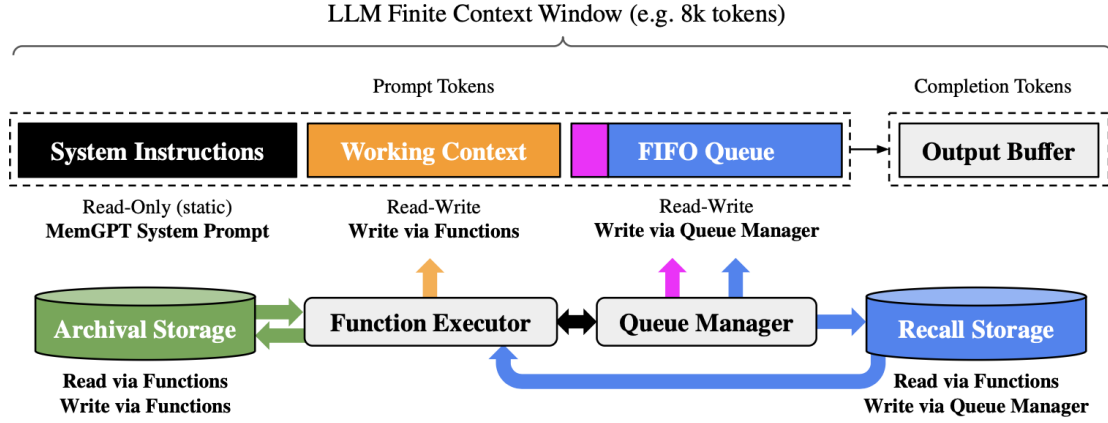


Figure 3.1: MemGPT memory architecture. The LLM’s finite context window is divided into system instructions, working context, and a FIFO queue, while older or less immediately relevant information is moved into external archival and recall storage. Through the function executor and queue manager, the agent can explicitly read, write, and retrieve memory, turning memory management into a system-level mechanism rather than relying on a single flat prompt.

is currently salient. The FIFO queue preserves local continuity in dialogue. Archival storage keeps durable facts. Recall storage preserves older conversational traces that may still matter later. Instead of forcing all of these roles into one undifferentiated prompt, MemGPT gives them separate places.

Table 3.1: Memory Layers in MemGPT

Layer	Role	Typical Content
System instructions	Persistent behavioral frame	Stable rules, role, high-level policies
Working context	Editable active state	Current goals, salient facts, state variables
FIFO queue	Recent interaction continuity	Latest dialogue turns, recent tool outputs
Archival storage	Durable long-term memory	User facts, reference documents, persistent knowledge
Recall storage	Older conversational history	Past dialogue segments that may later be relevant

3.4 Why Explicit Memory Operations Matter

A central design choice is that memory operations are exposed to the model through functions. The agent can decide to write information into memory, retrieve relevant past information, search external stores, or update existing entries. In this sense,

MemGPT is not just a retrieval-augmented generation system. It gives the agent some control over its own memory management.

This explicitness matters because memory is not only about storage volume. It is also about control policy. A flat long-context strategy assumes that once information is present somewhere in the prompt, the model will use it appropriately. MemGPT rejects this assumption. It instead requires the system to act on memory: to write, retrieve, and revise in targeted ways.

There is also an interpretability advantage. Because memory operations are explicit, humans can reason about why a certain piece of information was stored or retrieved. This makes the system easier to audit than a purely implicit long-context pipeline. For long-term assistants or agents that interact with sensitive user information, this explicitness is especially important.

3.5 Memory Update and Retrieval

One important property of useful agent memory is that it must be revisable. A simple append-only log is not enough. User information may change over time. A person who was once the user’s boyfriend may later become an ex-boyfriend. A user’s preference may change. A project status may become outdated. If the agent only appends new facts, it may later retrieve conflicting information.

MemGPT addresses this by allowing memory writes and updates. The agent can store compact facts in the working context or archival memory. When new information modifies older information, the memory state can be changed rather than merely expanded. This is important because memory should represent the current state of the world as well as past events.

Retrieval is equally important. Storing information is only useful if the agent can find it when needed. MemGPT allows the agent to search archival storage and recall storage. For example, if the user asks about a topic discussed in a previous session, the agent can issue a memory search and bring relevant information back into the active prompt. This explicit retrieval mechanism helps the model answer questions that would otherwise be impossible because the necessary information is outside the current context.

The main lesson is that memory is not just storage. It is a dynamic system involving write, update, search, and retrieval operations. A strong agent memory must be state-aware: it must distinguish between current facts, outdated facts, and historical conversation records.

3.6 Empirical Evidence: Deep Memory Retrieval

MemGPT is evaluated on tasks where standard fixed-context models struggle. One important evaluation is deep memory retrieval, where the agent must answer questions about topics from prior sessions. Without an external memory mechanism, a model cannot reliably answer if the relevant information is no longer in the active prompt.

The reported results show large improvements when MemGPT is used. For GPT-4 Turbo, the deep memory retrieval accuracy increases from approximately 35.3% without MemGPT to 93.4% with MemGPT. For GPT-4, the accuracy increases from approximately 32.1% to 92.5%. These results support the central claim that long-horizon agents need a memory system rather than relying only on the model’s standard context window.

Table 3.2: Deep Memory Retrieval Results Reported for MemGPT

Model	Without MemGPT	With MemGPT	Improvement
GPT-4 Turbo	35.3%	93.4%	+58.1
GPT-4	32.1%	92.5%	+60.4

The empirical takeaway is straightforward: when the task requires information from prior sessions, memory management can transform agent performance. The improvement is not because the underlying model suddenly becomes more intelligent in a general sense. Instead, the system architecture allows the model to access the right information at the right time.

3.7 Benefits and Potential Failure Cases

MemGPT has several clear benefits. First, it allows agents to scale beyond the active prompt in a principled way. Second, it makes memory operations explicit and inspectable. Third, it allows revisable state rather than forcing all information into an append-only transcript. Fourth, it supports long-term conversational continuity.

At the same time, memory management does not eliminate all failure modes. Retrieval may miss the relevant entry. The working context may still become cluttered if too much is retrieved. Memory entries may become stale if updates are incomplete. The agent may also store information at the wrong level of abstraction, preserving local details without extracting reusable procedural guidance.

These failure modes are significant because they mark the limits of memory-first design. Even if memory access is handled well, the agent still needs mechanisms

for transforming recurring experience into strategy. This naturally motivates a transition from memory architecture to context evolution.

3.8 Comparison with Naive Long-Context Bases

It is tempting to argue that the value of MemGPT will disappear as context windows become larger. However, a larger context window does not fully solve the underlying problem. First, the cost of placing all history in context can still be high. Second, longer context can dilute relevance. Third, a long undifferentiated prompt still does not offer explicit memory operations or revisable long-term state.

The difference between “all history in context” and “managed memory” is therefore structural, not merely quantitative. A managed memory system acknowledges that some information should be active, some archived, some recalled selectively, and some revised. This remains true even if raw context budgets continue to grow.

3.9 Summary and Evaluation

MemGPT represents the first stage of agent evolution studied in this report: memory-managed agents. Its key contribution is to treat memory as a system design problem. Instead of forcing all information into one prompt, it introduces a hierarchy of active context, working memory, archival storage, and recall storage. This allows agents to handle longer interactions and retrieve information from past sessions.

In my view, the most valuable aspect of MemGPT is that it makes memory operations explicit. The agent is not simply given a longer prompt; it is given mechanisms to write, update, and retrieve information. This makes the system more controllable and interpretable than a flat long-context approach. It also matches a practical intuition from using LLM agents: the problem is often not that the model is unable to reason, but that the right information is not available in the right form at the right moment.

At the same time, I see MemGPT as solving an access problem rather than a learning problem. It helps the agent access past information, but it does not automatically tell the agent how to convert that information into better strategies. A memory entry may say what happened, but it may not explain what rule should be followed next time. Therefore, memory is necessary for long-horizon agents, but not sufficient for continual improvement.

This limitation leads naturally to the next stage. Remembering facts is not the same as organizing experience into strategy. An agent may store many memories but still fail to convert them into reusable procedural knowledge. To solve tasks more effectively, the agent needs not only memory but also structured context. This leads to the second system: Agentic Context Engineering.

Chapter 4

Context Evolution: Agentic Context Engineering

4.1 Motivation: Why Raw Memory Is Not Enough

MemGPT shows that agents need memory beyond the active context window. However, memory alone does not solve the full problem of long-term agent improvement. A memory system can store facts, conversation history, and documents, but task performance often depends on procedural knowledge: how to solve a type of task, what mistakes to avoid, which tools to use, and how to interpret feedback.

If an agent simply stores all past interactions, the memory may become large, noisy, and difficult to use. A raw trajectory contains many details that are not reusable. It may include one-off names, temporary states, failed attempts, irrelevant dialogue, and environment-specific artifacts. The agent must somehow transform this experience into a form that helps future decisions.

A natural solution is summarization. The system can periodically compress the history into a shorter context. However, ACE argues that repeated summarization is risky. A shorter context is not necessarily a better context. Important details may be removed. Procedural knowledge may be flattened into vague statements. Repeated rewriting can gradually erode the information needed for good performance.

This motivates the main idea of ACE: context should not be treated as a static prompt or repeatedly compressed summary. Instead, it should become an evolving playbook. The playbook accumulates strategies, rules, templates, and troubleshooting knowledge over time. It is not raw memory; it is organized experience.

4.2 Context Collapse

ACE identifies a failure mode called context collapse. Context collapse occurs when iterative rewriting makes the context shorter or cleaner but less useful. This can happen when a system repeatedly summarizes a growing history into a compact form. Each summary may discard some details. Over many iterations, the context may lose the specific procedural knowledge that originally made it valuable.

This failure mode is important because many agent systems rely on summarization to handle long histories. Summarization is attractive because it controls token length. But it may introduce a brevity bias: the system prefers concise statements even when detailed knowledge is necessary. For example, a troubleshooting rule may need a specific trigger condition and a specific fix. If summarization turns it into a generic sentence such as “check the tool output carefully,” the resulting context becomes less actionable.

Context collapse also shows why context engineering is different from simple compression. The goal is not only to reduce length. The goal is to preserve and organize information that improves future performance. Good context should be structured, reusable, and specific enough to guide action.

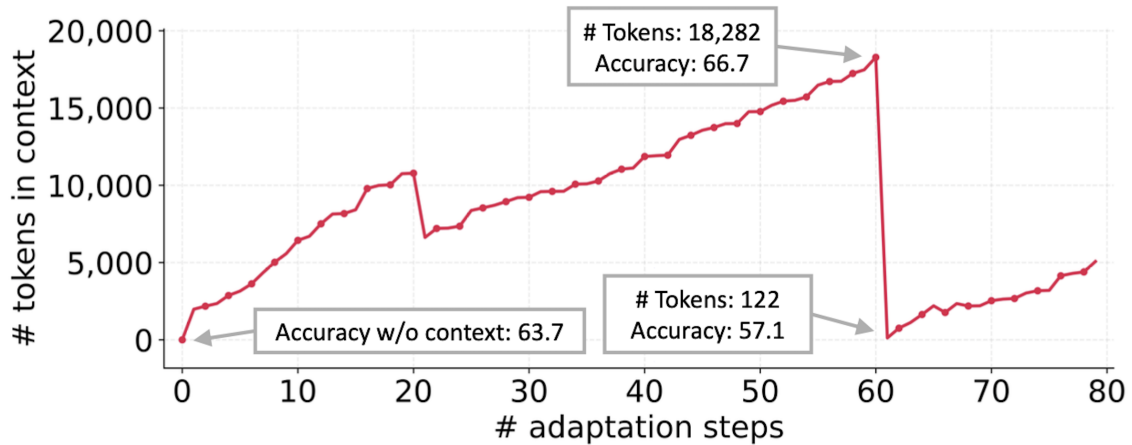


Figure 4.1: Context collapse under repeated monolithic rewriting. Although summarization can reduce the number of tokens in the context, repeated rewriting may remove useful procedural details and cause performance to drop. This motivates ACE’s argument that good context should not merely be shorter, but should preserve reusable knowledge in a structured and incrementally maintained form.

4.3 Why Summarization Alone Fails

Summarization fails in this setting for at least three reasons. First, summaries tend to compress away conditions. A procedural rule such as “when test output

changes but the error persists, inspect hidden assumptions in the earlier step” may be reduced to something too generic to guide future action. Second, summaries tend to flatten structure. They turn a trajectory with branching points, feedback loops, and local corrections into a single paragraph. Third, summaries may overfit to brevity rather than actionability.

This does not mean summarization has no value. A summary can still be useful as a coarse overview. The problem is that repeated summarization is being asked to serve as a substitute for structured knowledge construction. ACE rejects that substitution. It argues that if the goal is future performance, the system must preserve reusable details instead of repeatedly sacrificing them for shorter prose.

4.4 The Generator–Reflector–Curator Framework

ACE avoids context collapse by updating context through a modular process. The framework contains three main components: Generator, Reflector, and Curator.

The **Generator** uses the current context or playbook to solve tasks. During this process, it produces trajectories that include actions, intermediate reasoning, outputs, and feedback. These trajectories provide evidence about what the current context helps the model do and where it fails.

The **Reflector** analyzes the trajectories. It identifies what worked, what failed, and what lessons can be extracted. Instead of treating a trajectory as raw text to be stored, the Reflector converts it into higher-level insights. These insights may include successful strategies, failure patterns, missing instructions, or corrections.

The **Curator** updates the playbook. It converts reflected insights into structured delta context items. A delta item is an incremental update rather than a full rewrite of the entire context. This is a key design choice. By updating the playbook incrementally, ACE avoids the destructive effects of repeatedly rewriting everything.

The conceptual contribution of this framework is that context becomes an evolving external knowledge structure. It is not merely a prompt written once by a human. It is not a raw memory log. It is not a compressed summary. It is a maintained playbook that changes as the agent gains experience.

4.5 From Trajectories to Reusable Playbook Items

The transformation from trajectory to playbook item is one of the most important steps in ACE. A trajectory contains specific action sequences and local environment

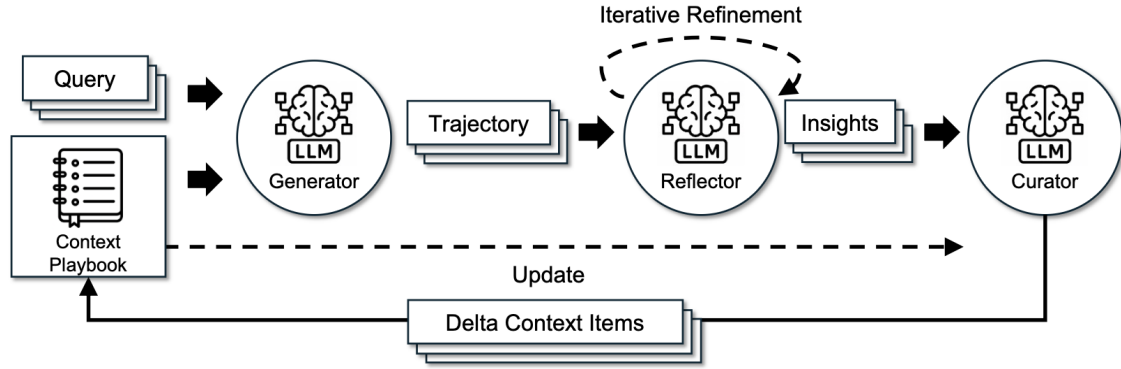


Figure 4.2: Agentic Context Engineering framework. The generator uses the current context playbook to solve tasks and produce trajectories; the reflector analyzes these trajectories to extract useful insights; and the curator converts these insights into delta context items that update the playbook. This loop turns raw interaction experience into structured and reusable external strategy.

details. A playbook item must abstract away from those specifics while preserving the procedure that transfers. This requires a change in representation.

A good playbook item is neither a full transcript nor a vague slogan. It should preserve triggers, decision points, common pitfalls, and concrete remedies. If the item becomes too specific, it will not generalize. If it becomes too general, it will not help the next trajectory. This middle layer of abstraction is what makes the playbook useful.

This is also why the reflector and curator are separate roles. Reflection identifies the lesson; curation determines how that lesson should be represented and merged into the existing playbook. Without curation, the system would accumulate raw or redundant items. Without reflection, curation would have no principled input.

4.6 Structured Playbooks as External Strategy

The output of ACE is a structured playbook. This playbook may contain strategies, hard rules, code snippets, templates, troubleshooting advice, and domain-specific procedures. The important point is that the playbook represents reusable procedural knowledge rather than raw history.

For example, in an agent task, the playbook may specify when to call a tool, how to verify the result, and how to recover from common errors. In a finance reasoning task, it may include formulas, data-processing templates, or domain-specific caveats. In a coding task, it may include reusable snippets or debugging procedures.

This is different from MemGPT-style memory. MemGPT might store that a user said something or that a document contained a fact. ACE tries to store how to

act better next time. In this sense, ACE externalizes strategy. The model still has fixed weights, but it receives a stronger external context that guides its behavior.

A structured playbook has several advantages. First, it reduces noise by separating reusable knowledge from one-off details. Second, it supports incremental improvement because new lessons can be added as delta updates. Third, it makes the agent’s accumulated knowledge more interpretable. A human can inspect the playbook and understand what strategies the agent is using.

Table 4.1: Desirable Properties of Playbook Items

Property	Why It Matters
Reusable	The item should transfer beyond a single trajectory
Procedural	The item should guide action, not merely restate a fact
Specific	The item should contain enough detail to be applied
Compact	The item should not waste context with irrelevant local detail
Editable	The item should be easy to inspect, revise, or remove

4.7 Empirical Results and Limitations

ACE reports strong empirical gains across agent and domain-specific reasoning tasks. The paper reports average improvements of approximately 10.6% on AppWorld and 8.6% on finance tasks. These gains suggest that better context construction can substantially improve agent behavior even when the underlying model is not fine-tuned.

Table 4.2: Summary of ACE Performance Improvements

Benchmark / Domain	Average Improvement
AppWorld	+10.6%
Finance tasks	+8.6%

The significance of these results is that context itself can be optimized. Instead of assuming that model weights are the only location of intelligence, ACE shows that external context can function as an evolving knowledge substrate. This is especially attractive because context updates are cheaper and more flexible than model training. They can be inspected, edited, and deployed quickly.

At the same time, ACE has an important boundary. It mainly improves the external context. The underlying policy remains mostly fixed. The model must still read the playbook, identify relevant guidance, and apply it correctly. If the playbook becomes too large or complex, the model may fail to use it effectively. Also, context adaptation may not be enough when the required improvement involves deeper behavioral changes or new capabilities that are difficult to express as external instructions.

4.8 Playbook Growth, Redundancy, and Conflict

However, structured playbooks also introduce challenges. If the playbook grows too large, the model may struggle to select relevant items. If rules overlap or conflict, the agent may become confused. If the playbook contains outdated strategies, performance may degrade. Therefore, playbook management requires not only adding information but also organizing, revising, and pruning it.

This is one of the main reasons ACE is interesting but incomplete. It correctly identifies the problem of context collapse, but the success of the playbook also depends on long-term maintenance. A playbook that keeps growing without validation or de-duplication may eventually become another form of context overload.

This limitation motivates the next stage of agent evolution. If external context cannot fully solve the problem, then the agent may need to update its policy through training. This is where OpenClaw-RL enters the picture.

4.9 Summary and Evaluation

ACE represents the second stage of agent evolution: context-evolving agents. Its key contribution is to move beyond memory storage toward structured experience organization. The system argues that good context is not merely shorter context; it is better organized, more reusable, and less vulnerable to collapse.

Among the three papers, ACE is the closest to my own understanding of how experience should be reused. What I find most convincing is that ACE does not treat past trajectories as content to be saved directly. Instead, it treats them as evidence from which reusable procedural knowledge can be extracted. This distinction is important. A raw trajectory is usually too specific, but a well-written playbook item can transfer to future tasks.

However, my main concern is that ACE may face a new version of the context problem as the playbook grows. A structured playbook is better than a raw history,

but it can still become too large, redundant, or internally inconsistent. If many rules are added over time, the model may struggle to select the right ones. Some rules may overlap, and some may contradict each other. Therefore, context evolution needs not only generation and curation, but also validation, pruning, and conflict resolution.

Compared with MemGPT, ACE focuses less on retrieving past facts and more on constructing future-useful procedural knowledge. MemGPT externalizes memory, while ACE externalizes strategy. However, ACE still relies on a fixed underlying model. This raises the next question: can agents improve not only their external context but also their internal behavior? The next chapter analyzes OpenClaw-RL as a response to this question.

Chapter 5

Policy Evolution: OpenClaw-RL

5.1 Motivation: From Evolving Context to Evolving Behavior

MemGPT and ACE both improve agents without directly changing model weights. MemGPT provides better memory access. ACE provides better external strategy. These approaches are powerful because they are flexible and relatively cheap. However, they also share a limitation: the underlying model policy remains mostly fixed.

This limitation becomes important as external context grows. A highly capable agent may already have a long system prompt, many built-in tools, and a large set of skills or rules. Adding more context does not always help. Some skills may be redundant. Some may conflict. Some may be relevant only in rare situations. The model must select and apply the right guidance, but this selection process itself may be unreliable.

Therefore, the deeper problem is not only how to add better context. The model itself may need to become better at using experience. It may need to internalize patterns from interaction so that good behavior becomes part of the policy, not only an instruction in the prompt.

OpenClaw-RL addresses this problem by treating interaction as a training source. The system starts from a simple observation: every agent interaction produces a next-state signal. After the agent acts, something happens. A user replies, a tool returns output, a terminal command produces stdout or stderr, a GUI changes state, or a test suite passes or fails. These next-state signals contain information about whether the action was good and how it could be improved.

5.2 Why External Context Eventually Hits a Limit

The transition from ACE to OpenClaw-RL is motivated by a practical limit of external context. External knowledge can help only if the model can reliably select and apply it. As the amount of external strategy grows, the agent may face overlapping rules, buried priorities, and conflicting heuristics. At that point, the failure is no longer only about missing information. It is about the model’s inability to use information consistently.

This matters because some improvements are naturally behavioral rather than descriptive. For instance, an agent may need to become more conservative when using a dangerous tool, more systematic when diagnosing test failures, or more adaptive to a user’s preferred style. These are repeated patterns of action selection. They can be described externally, but sometimes they are more effective when internalized into the policy.

OpenClaw-RL therefore takes the next step. Instead of only improving the external context, it uses interaction itself to improve the policy.

5.3 Train-While-Serving Architecture

The key design principle of OpenClaw-RL is “train while serving.” The agent continues to serve live requests while the training system collects interaction data and updates the policy asynchronously. This is important because deployed agents cannot stop functioning every time the model needs to improve.

The architecture can be understood as three connected parts. First, there are agent environments. These may include personal assistants, terminal agents, GUI agents, software engineering agents, and tool-call agents. Each environment produces interaction trajectories.

Second, environment servers route interactions and collect data. They serve as the interface between deployed agents and the training infrastructure. They collect next-state signals such as user replies, tool outputs, terminal results, GUI observations, and test verdicts.

Third, the training loop updates the policy. A policy server handles inference, while a reinforcement learning engine and reward modeling components process collected data. The system is asynchronous: policy serving, environment interaction, reward evaluation, and training are decoupled.

This architecture is ambitious because it treats deployed interaction not merely as usage but as a learning opportunity. Instead of manually collecting a fixed train-

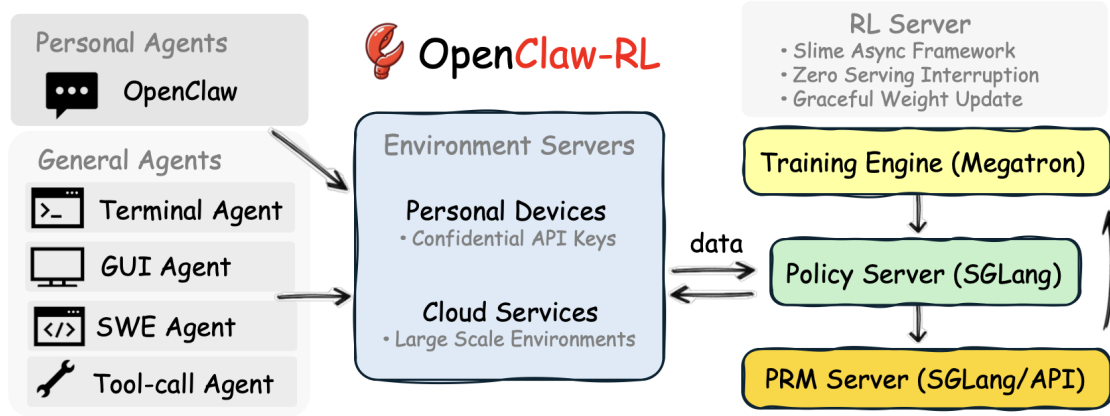


Figure 5.1: OpenClaw-RL system architecture for train-while-serving agents. Personal and general agents interact with users, tools, and environments through environment servers, while the RL server asynchronously collects interaction data and updates the policy. This design allows deployed agents to continue serving requests while their behavior is improved through reinforcement learning and reward modeling.

ing dataset, the system continuously receives new signals from real agent behavior.

5.4 Next-State Signals as a General Learning Interface

A central insight of OpenClaw-RL is that diverse agent settings can be described in a unified way through next-state signals. After an agent takes an action, the world changes. The user replies, the tool returns something, the terminal prints an error, the GUI changes, or the test suite reports success or failure. These next states are not identical, but they share a common structural role: they carry information about action quality and future opportunity.

This view is powerful because it unifies heterogeneous environments under one training interface. The system does not require every environment to look like a classical simulator with a handcrafted scalar reward at every step. Instead, it interprets the consequences of action as potential supervision. That supervision can be converted into reward, correction, or intermediate estimates.

By treating next-state signals as the common substrate, OpenClaw-RL broadens what counts as learnable interaction. Everyday conversation, tool usage, terminal commands, and software engineering traces all become potential data for continual improvement.

5.5 Learning Signals from Interaction

OpenClaw-RL uses multiple forms of learning signal. The first is **binary reward**. In many environments, the system can observe whether an outcome is good or bad. A terminal command may succeed or fail. A test may pass or fail. A user may accept or reject an answer. This produces a scalar reward that can be used for reinforcement learning.

The second is **directional feedback**. A scalar reward tells the agent whether an action was good, but it may not tell the agent how to improve. Directional feedback provides richer information, such as hints, corrections, or explanations. OpenClaw-RL uses on-policy distillation to convert such feedback into learning signals that guide the policy toward better actions.

The third is **step-wise reward**. Long trajectories are difficult to train from final outcomes alone because credit assignment becomes sparse. If a coding agent fails after many steps, it is hard to know which earlier action caused the failure. Step-wise reward provides intermediate estimates so that learning becomes denser and more stable.

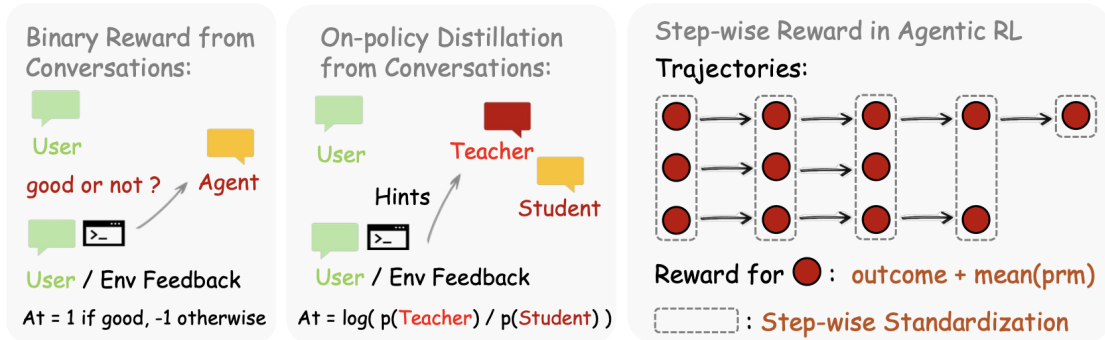


Figure 5.2: Learning signals used by OpenClaw-RL. The framework supports binary rewards from user or environment feedback, directional supervision through on-policy distillation, and step-wise reward estimation for long trajectories. These complementary signals allow interaction data to provide both outcome-level evaluation and more detailed guidance for policy improvement.

This combination is important because agent environments are diverse. Some settings provide clear verifiable rewards, such as tests or exit codes. Others provide softer feedback, such as user corrections. A general agent learning system must be able to use multiple signal types.

5.6 Learning Across Agent Environments

OpenClaw-RL is designed to work across multiple agent settings. This is one of its central claims: personal conversations, terminal tasks, GUI tasks, software engineering tasks, and tool-call traces are not completely separate learning problems. They all produce next-state signals that can be used to improve the policy.

In **personal assistant** settings, the next-state signal may be the user’s reply. If the user corrects the agent, asks the question again, or expresses dissatisfaction, this provides feedback. If the user continues smoothly, this may indicate that the response was useful. Personalization can therefore be learned from ordinary interaction.

In **terminal** settings, the next-state signal may include stdout, stderr, and exit codes. These signals are often precise and automatic. If a command fails, the error message may indicate what went wrong. If a script succeeds, the exit code provides a clear signal.

In **GUI** settings, the next-state signal may be a visual state change or task progress. The agent may click a button, observe the new interface state, and infer whether the action moved toward the goal.

In **software engineering** settings, the next-state signal may include test results, diffs, linter outputs, and compiler errors. These signals are especially useful because they provide both outcome feedback and debugging information.

In **tool-call** settings, the next-state signal is the tool output. The agent can learn whether it used the right tool, passed the right arguments, and interpreted the result correctly.

Table 5.1: Next-State Signals Across Agent Settings

Setting	Next-State Signal	Possible Reward	Possible Directive Feedback
Personal assistant	User replies and corrections	Acceptance or dissatisfaction	Preference correction
Terminal	stdout, stderr, exit codes	Command success or failure	Error messages
GUI	Visual state changes	Task progress	Interface observations
Software engineering	Tests, diffs, lint outputs	Pass/fail verdicts	Compiler or test errors
Tool-call agents	Tool outputs	Correctness of call result	Tool output interpretation

The significance of this unified view is that it reframes agent deployment. Every

interaction becomes potential training data. The agent is not only solving tasks; it is generating experience that can improve future behavior.

5.7 Empirical Implications and Personalization

The paper reports scalable reinforcement learning results across multiple agent settings, including terminal, GUI, software engineering, and tool-call environments. The reported curves generally improve as RL training proceeds. This supports the claim that interaction data can be converted into useful learning signals.

The system also includes a personalization example. In one setting, the desired student response should sound less AI-like. In another, the desired teacher response should be more specific and friendly. After a small number of interactions, the personalization score improves substantially. This example is important because it shows a use case where the objective is not only task correctness but also adaptation to user preference.

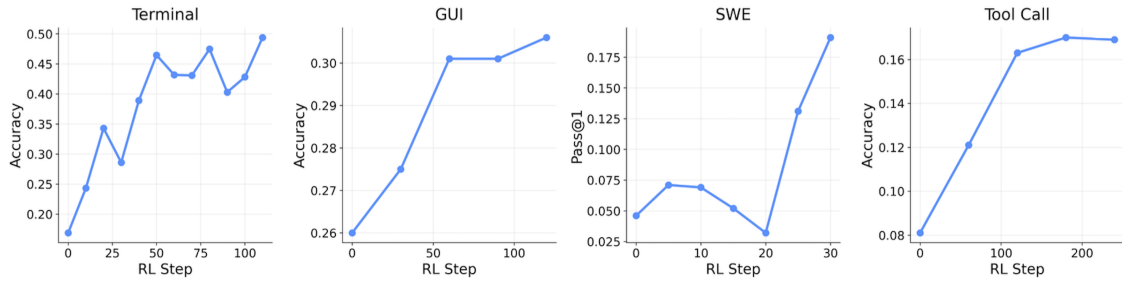


Figure 5.3: Performance improvement of OpenClaw-RL across multiple agent settings. The curves show that reinforcement learning from interaction can improve agents in terminal, GUI, software engineering, and tool-call environments. This supports the broader claim that agent behavior can be optimized using feedback generated during deployment rather than only through static pretraining or prompt engineering.

The broader implication is that agents may improve simply by being used. This is a major shift from static deployment. In traditional deployment, a model is trained, released, and then used. In OpenClaw-RL’s vision, deployment itself becomes part of training. The boundary between inference and learning becomes less rigid.

5.8 Safety, Stability, and Deployment Challenges

However, this also introduces serious challenges. Training from live interaction requires careful safety controls. User feedback can be noisy or ambiguous. Automated

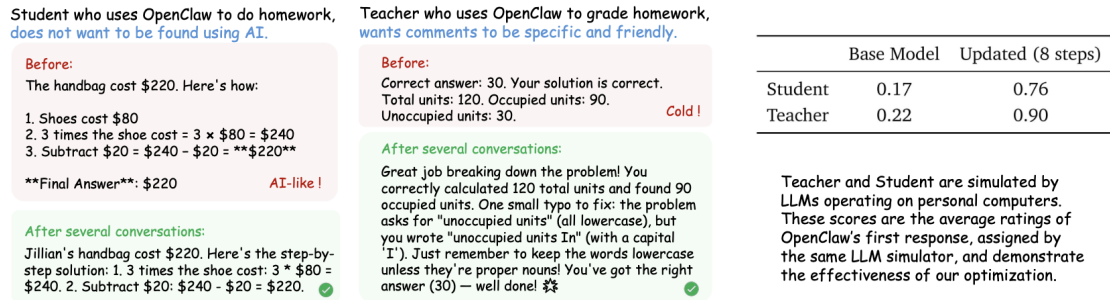


Figure 5.4: Personalization example in OpenClaw-RL. After several interactions, the agent adapts to different user preferences: the student setting encourages responses that sound less AI-like, while the teacher setting encourages more specific and friendly feedback. This example illustrates how interaction-based learning can improve not only task correctness, but also user-specific behavior and communication style.

rewards can be biased or hackable. Asynchronous updates can introduce instability. Policy updates may improve one behavior while harming another. Therefore, OpenClaw-RL points toward a powerful future but also raises difficult engineering and alignment questions.

There is also a reversibility challenge. If a playbook rule is bad, it can often be edited directly. If a policy update is bad, rollback is more complicated. This makes policy learning simultaneously powerful and risky. It can internalize good behavior, but it can also internalize the wrong lesson from low-quality feedback.

5.9 Summary and Evaluation

OpenClaw-RL represents the third stage of agent evolution: policy-evolving agents. Unlike MemGPT and ACE, it does not only modify memory or context. It uses interaction data to update the model's behavior. Its central insight is that next-state signals are universal across agent settings and can be used as training signals.

I see OpenClaw-RL as the most ambitious of the three systems because it attempts to move improvement from the prompt into the policy itself. This is attractive because some behaviors are difficult to fix with external instructions alone. For example, if an agent repeatedly misuses tools, ignores feedback, or fails to adapt its style to the user, adding another rule to the prompt may not be enough. Training the policy can make the improved behavior more natural and consistent.

However, OpenClaw-RL is also the most difficult system to deploy safely. Updating model weights is less transparent than editing a memory entry or playbook item. It is also harder to reverse. If the reward signal is noisy or biased, the policy

may learn the wrong behavior. If the training distribution is narrow, the agent may improve on one setting but regress on another. Therefore, I do not view policy learning as a replacement for memory or context engineering. Instead, I view it as a deeper adaptation layer that should be used carefully when external mechanisms are not enough.

Compared with ACE, OpenClaw-RL moves from external strategy to internal policy improvement. Compared with MemGPT, it moves from remembering past information to learning from consequences. This makes it a natural final stage in the evolution path studied in this report, but not necessarily the only or always preferred solution.

Chapter 6

Comparative Analysis

6.1 What Is Improved: Memory, Context, or Policy

The three systems studied in this report improve different parts of the agent stack. MemGPT improves memory access and storage. ACE improves external context and strategy. OpenClaw-RL improves policy behavior.

MemGPT asks: **How can agents remember?** Its answer is hierarchical memory and virtual context management. The main bottleneck is that relevant information may be outside the active context window. The solution is to give the agent explicit memory operations.

ACE asks: **How can context evolve?** Its answer is structured playbooks updated through generation, reflection, and curation. The main bottleneck is that raw memory or repeated summaries do not produce reliable procedural guidance. The solution is to transform experience into reusable external strategy.

OpenClaw-RL asks: **How can agents learn from interaction?** Its answer is asynchronous reinforcement learning and on-policy distillation from next-state signals. The main bottleneck is that external context alone cannot fully update behavior. The solution is to train the policy using feedback from deployment.

This comparison shows that the systems are complementary rather than competing. A strong future agent may use MemGPT-like memory to store and retrieve information, ACE-like playbooks to organize experience, and OpenClaw-RL-like training to update behavior.

Table 6.1: Comparison of Three Agent Evolution Systems

System	Main Question	What Is Improved	Knowledge Form	Adaptation Style	Main Limitation
MemGPT	How can agents remember?	Memory access and storage	Hierarchical memory	Memory management	Remembers facts but does not deeply organize strategy
ACE	How can context evolve?	External playbook / context	Structured procedural knowledge	Reflection and curation	Policy remains mostly fixed
OpenClaw-RL	How can agents learn from interaction?	Policy / behavior	Rewarded and distilled trajectories	Reinforcement learning from on-going interaction	Requires reliable feedback and safe training

6.2 Knowledge Representation Across the Three Systems

The systems also differ in how they represent knowledge.

In MemGPT, knowledge is represented as memory entries. These entries may be facts about the user, older conversation records, or archived information. The representation is useful for retrieval but may not directly encode procedural strategies.

In ACE, knowledge is represented as structured playbook items. These items are closer to rules, strategies, templates, and troubleshooting notes. They are designed to guide future behavior, not merely record past facts.

In OpenClaw-RL, knowledge is represented partly in training trajectories and partly in the updated model parameters. The final learned behavior may be less directly interpretable than a playbook, but it can be more deeply internalized into the policy.

This creates a tradeoff between interpretability and integration. External memory and playbooks are more inspectable and easier to edit. Internal policy updates may produce smoother behavior but are harder to inspect and control. A practical system may need both.

6.3 Adaptation Mechanisms

The adaptation mechanisms also differ.

MemGPT adapts at inference time through memory operations. It decides what to retrieve, what to store, and what to update. This is fast and flexible, but it does not necessarily improve the model’s future reasoning unless the right memory is retrieved again.

ACE adapts through context updates. It analyzes trajectories and modifies

the playbook. This creates more reusable knowledge than raw memory, but still depends on the model’s ability to read and apply the playbook.

OpenClaw-RL adapts through model training. It collects interaction signals and updates the policy. This can improve behavior even without explicitly inserting a rule into the prompt. However, it is more expensive and riskier than context updates.

The key design question is when to use each mechanism. Some information should remain in memory, such as user preferences and personal facts. Some experience should become external context, such as task strategies and common pitfalls. Some behavior may need to be learned by the policy, especially when it involves repeated patterns of action selection or style adaptation.

6.4 External vs Internal Adaptation

The distinction between external and internal adaptation becomes especially clear when the three systems are placed side by side. MemGPT and ACE mainly operate externally: they alter what the model sees, not what the model is. OpenClaw-RL operates more internally by attempting to change what the model will do in the future even without re-supplying the same guidance.

External adaptation is usually easier to inspect and correct. A memory entry can be deleted. A playbook rule can be rewritten. Internal adaptation is harder to inspect, but can produce smoother and more persistent behavior. This suggests a layered strategy rather than a winner-takes-all strategy. Some knowledge should remain external because it is dynamic or sensitive. Other knowledge may be internalized when it reflects stable repeated patterns.

6.5 When Should an Agent Update Memory, Context, or Policy?

A useful way to synthesize the three papers is to ask not only what each improves, but when each improvement type is appropriate. If the system encounters a fact about the user or environment that may later be needed, memory is often the right target. If the system encounters a recurring procedure or failure mode that should be available in interpretable textual form, context or playbook updates are often the right target. If the system repeatedly exhibits a deeper behavioral weakness, policy learning may be the right target.

This leads to a practical heuristic:

- **Update memory** when the experience is primarily factual, stateful, or personal.
- **Update context/playbook** when the experience encodes reusable procedure, troubleshooting advice, or structured strategy.
- **Update policy** when the weakness is behavioral, repeated, and difficult to fix reliably through external instructions alone.

Table 6.2: A Simple Decision Framework for Agent Updates

Type of New Experience	Best Initial Target	Reason
User preference or factual state	Memory	Dynamic information should remain editable and retrievable
Reusable troubleshooting routine	Context / playbook	Procedural knowledge should remain inspectable and reusable
Repeated behavior pattern	Policy	Stable action bias may require internalized improvement

6.6 Critical Comparison

The three systems are all important, but they are convincing in different ways and incomplete in different ways.

MemGPT is convincing because it makes memory explicit, controllable, and system-level. It avoids the naive idea that all useful information should simply be placed inside the prompt. Its limitation is that it mainly improves access to information, not the transformation of experience into strategy.

ACE is convincing because it treats experience as something that should be processed before reuse. It does not simply store trajectories; it extracts structured procedural knowledge. Its limitation is that the playbook itself can become a new source of complexity. Without strong validation and pruning, the playbook may grow into another long-context problem.

OpenClaw-RL is convincing because it recognizes that real interaction produces learning signals. It is the most direct attempt to make agents improve through

use. Its limitation is that policy updates are harder to inspect, harder to control, and more dependent on feedback quality.

This comparison leads to an important conclusion: the three systems should not be understood as replacing one another. Instead, they occupy different layers of adaptation. Memory is useful for facts and state. Playbooks are useful for reusable strategies. Policy learning is useful for repeated behavioral patterns that should become internal to the model.

6.7 Lessons Learned from the Three Papers

From comparing the three systems, I draw four main lessons.

Lesson 1: Bigger context is not the same as better context. Long context can help, but it can also bury useful information. MemGPT and ACE both show that the structure of context matters as much as its size.

Lesson 2: Raw experience must be transformed before it becomes useful. A trajectory is not automatically a skill. It contains useful evidence, but also noise and task-specific details. ACE is important because it explicitly transforms trajectories into reusable playbook items.

Lesson 3: External memory and internal learning solve different problems. Memory and context engineering are transparent, editable, and relatively safe. Policy learning is more powerful but less interpretable. A practical agent should decide which adaptation layer is appropriate for each type of knowledge.

Lesson 4: Stability is the hidden challenge of self-improving agents. It is easy to imagine an agent that keeps updating itself, but much harder to make the updates reliable. Any self-improving system must prevent regressions, resolve conflicts, and avoid learning from bad feedback.

The order of the three papers is also meaningful. If one starts directly from OpenClaw-RL, the problem may look like a reinforcement learning problem. But by first studying MemGPT and ACE, it becomes clear that many agent failures may be solved without changing model weights. This makes the final question more precise: when is external adaptation enough, and when do we need policy learning?

6.8 Stability as the Central Bottleneck

A recurring theme across all three systems is stability. Memory retrieval must be stable enough not to miss important facts or surface stale facts. Context evolution must be stable enough not to collapse useful knowledge into vague summaries or conflicting rules. Policy learning must be stable enough not to overfit to noisy rewards or regress on previously good behavior.

This is why self-improving agents are harder than they first appear. The problem is not simply to keep adding updates. It is to keep adding updates without making the system worse. In that sense, stability is the hidden systems problem connecting memory, context, and learning.

6.9 Limitations and Open Challenges

Although the three systems represent important progress, many challenges remain.

First, **memory retrieval is still difficult**. A memory system must decide what to retrieve and when. Retrieving too little causes forgetting. Retrieving too much causes context noise. Retrieval errors can also lead to incorrect or outdated information being used.

Second, **context growth can create conflicts**. Structured playbooks are better than raw logs, but they can still become large, redundant, or contradictory. A context-evolving agent needs mechanisms for pruning, merging, and validating knowledge.

Third, **policy learning from interaction can be unstable**. Reinforcement learning from live data may suffer from noisy rewards, distribution shift, reward hacking, and catastrophic forgetting. If an agent learns continuously, it must do so safely.

Fourth, **feedback quality is a central bottleneck**. OpenClaw-RL depends on next-state signals, but not all signals are equally reliable. User replies may be ambiguous. Tool outputs may be incomplete. Reward models may misjudge quality. Learning from poor feedback can make the agent worse.

Fifth, **the boundary between external context and internal learning is unresolved**. Updating context is cheaper, more interpretable, and easier to reverse. Updating weights may produce deeper improvement but is more expensive and harder to control. Future systems need principled methods for deciding which knowledge should be stored externally and which should be internalized.

Finally, **evaluation remains challenging**. Static benchmarks may not capture

long-term agent improvement. A self-improving agent should be evaluated over time, across repeated interactions, and under changing conditions. This requires new benchmarks and protocols for continual agent learning.

6.10 Summary

The comparative analysis shows that MemGPT, ACE, and OpenClaw-RL should be understood as complementary layers in a broader theory of agent evolution. MemGPT provides explicit memory management, ACE turns experience into reusable external strategy, and OpenClaw-RL moves toward policy-level adaptation. The broader lesson is that future agents will likely need to coordinate all three layers rather than choosing only one.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This report studied the evolution of AI agents from memory architecture to self-improving systems. The central argument is that future agents will not be defined only by their single-turn reasoning ability. They will also be defined by how well they remember, organize experience, and improve through use.

MemGPT represents the memory stage. It shows that fixed-context LLMs need explicit memory management to handle long-horizon interaction. By treating memory as an operating-system-like design problem, it introduces hierarchical memory and virtual context management.

ACE represents the context evolution stage. It shows that raw memory is not enough. Agents need structured, reusable procedural knowledge. By turning experience into evolving playbooks, ACE improves the external context that guides the model.

OpenClaw-RL represents the policy evolution stage. It shows that interaction can become training data. By using next-state signals from personal and general agent environments, it enables agents to improve their behavior through reinforcement learning and on-policy distillation.

Together, these systems define a clear progression: from storing information, to structuring knowledge, to updating behavior. This progression suggests that the next generation of AI agents will be dynamic systems rather than static models.

7.2 Toward a Unified Agent Architecture

My main conclusion from this report is that future agents should not choose between memory, context engineering, and reinforcement learning. A stronger design would combine them in a layered architecture, where each layer handles a different kind of adaptation.

One possible architecture would contain five layers. The first layer is **short-term working context**, which stores the current task state and recent interaction. The second layer is **long-term memory**, which stores user facts, prior sessions, and retrievable documents. The third layer is **structured playbook or skill memory**, which stores reusable procedures learned from experience. The fourth layer is a **validation mechanism**, which checks whether new memory entries, playbook rules, or policy updates actually improve performance without causing regressions. The fifth layer is a **policy learning loop**, which uses reliable interaction signals to update the model when external context is not enough.

In this view, MemGPT mainly contributes the memory layer, ACE contributes the playbook layer, and OpenClaw-RL contributes the policy learning layer. The missing piece is the control logic that decides where each new experience should go. Some experiences should be stored as facts. Some should become external rules. Some should become training data. A unified future agent must make this decision explicitly.

7.3 Implications for Future Agent Benchmarks

If agents are increasingly evaluated as systems that evolve over time, then traditional static benchmarks will be less informative on their own. A strong future benchmark should test whether the agent remembers earlier information, whether it turns past errors into improved strategy, and whether it can improve through repeated interaction without catastrophic regressions.

This implies at least three design needs. First, benchmarks should include streams of tasks rather than only isolated items. Second, they should measure not only final reward but also stability over time. Third, they should reveal whether external and internal adaptation are working together effectively. The future of evaluation is therefore likely to be more longitudinal, interactive, and systems-oriented.

7.4 Implications for Real-World Deployment

The practical implications are equally important. A real-world agent must deal with personalization, privacy, retrieval errors, outdated guidance, noisy feedback, and safe rollback. Memory systems raise questions about data retention and user control. Context evolution raises questions about rule conflict and maintenance. Policy learning raises questions about reward quality and deployment safety.

These are not secondary concerns. They are part of the core engineering problem of deploying self-improving systems responsibly. As agents become more adaptive, the need for auditing, validation, interpretability, and rollback becomes stronger, not weaker.

7.5 Future Work

Several future directions are especially important.

First, future agents need better integration between memory, context, and learning. Current systems often focus on one layer. A complete agent should decide whether a new experience should be stored as memory, converted into a playbook rule, or used for policy training.

Second, agents need stronger validation mechanisms. Before committing a new memory, context update, or policy change, the system should check whether it improves future performance without causing regressions. This is especially important for continual learning.

Third, safety and privacy must be central. Long-term memory may store sensitive user information. Policy learning from interaction may incorporate private or biased data. Future systems must include mechanisms for consent, deletion, auditing, and safe rollback.

Fourth, evaluation protocols should move beyond static benchmarks. A self-improving agent should be evaluated over a stream of interactions, with changing tasks, delayed feedback, and long-term dependencies. Metrics should capture not only final performance but also stability, memory accuracy, adaptation speed, and resistance to degradation.

Finally, future research should study the balance between external and internal adaptation. External context is transparent and editable, while internal policy learning is powerful but harder to inspect. The most effective agents may combine both: using memory and playbooks for flexible knowledge management, while training the policy to become better at selecting and applying that knowledge.

The evolution of AI agents is therefore not only about making models larger. It is about building systems that can live with experience. The long-term goal is an agent that remembers what matters, organizes what it learns, and improves safely through continued interaction.

7.6 Final Perspective on Hybrid Agent Systems

My own view after studying these three papers is that the future of agents will be hybrid. Memory systems will remain necessary because agents need state and personalization. Context engineering will remain necessary because experience must be transformed into usable external knowledge. Policy learning will also become important because some improvements should be internalized into behavior. The central research challenge is not choosing one of these directions, but coordinating them in a stable and interpretable way.

The best future agents may therefore look less like single monolithic models and more like layered systems. Their intelligence will be distributed across parameters, memory stores, evolving textual strategy, validation mechanisms, and training loops. The challenge for research is to make these layers cooperate without collapsing into noise or instability.

References

- [1] Tom B. Brown et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 2020.
- [2] Long Ouyang et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.
- [3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [4] John Schulman et al. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [5] Noah Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*, 2023.
- [6] Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. OpenClaw-RL: Train Any Agent Simply by Talking. *arXiv:2603.10165*, 2026.
- [7] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*, 2023.
- [8] Shunyu Yao et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Advances in Neural Information Processing Systems*, 2023.
- [9] Xiang Yue et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. *arXiv preprint*, 2024.
- [10] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. *arXiv:2510.04618*, 2025.